

PP1 - Vyhodnocování úplnosti testů vůči softwarovým požadavkům - formalismus

April 2026

1 The primitives

These primitive sets serve as the base that the rest of the formalism builds on. The primitive sets are the following:

- T_D, T_C – the *discrete time* and *continuous time* sets. An infinite totally ordered set; the time set can have *discrete* or *continuous* flavour; exactly one flavour is chosen per requirement.
- V – the *Values* set. These can be used to describe the values of entities.

Definition 1 (Discrete time set). *The discrete time set T_D is totally ordered and countable.*

$$T_D = \mathbb{N}$$

Definition 2 (Continuous time set). *The continuous time set T_C is totally ordered and uncountable.*

$$T_C = \mathbb{R}_{\geq 0}$$

Definition 3 (Duration). *A duration is a non-negative element of the time set, representing the length of a time span:*

$$\Delta = T$$

Thus $\Delta = \mathbb{N}$ for discrete time and $\Delta = \mathbb{R}_{\geq 0}$ for continuous time.

Definition 4 (Interval). *An interval is a pair of a start time point and a duration:*

$$\mathcal{I} = T \times \Delta$$

An interval $(t, d) \in \mathcal{I}$ represents the time span $[t, t + d]$.

Definition 5 (Values). *The values set V is the union of the following sets:*

$$V = \mathbb{B} \cup \mathbb{R} \cup T \cup \mathcal{I}$$

where $\mathbb{B} = \{\text{true}, \text{false}\}$ is the set of booleans, $\mathbb{R}$ is the set of real numbers, T is the time set, and $\mathcal{I}$ is the set of intervals.

2 Entities and entity types

An entity type is an item belonging to a set of predefined entity types.

Definition 6 (Entity type set).

$$T_E = \{\text{Signal}, \text{Storage}, \text{Event}, \text{Channel}, \text{State}, \text{Value}, \text{Abstract}\}$$

Definition 7 (Entity name set).

$$N_E \subseteq \Sigma^*$$

, where Σ is an alphabet.

Entity type modifiers are properties which can be attached to an entity of a particular type. Semantics are defined for each modifier.

Definition 8 (Entity type modifiers set).

$$M = \{m \mid m \text{ is a modifier of an entity type}\}$$

Each entity type is compatible with only a subset of the entity modifiers. Therefore we define the *entity type modifiers function*.

Definition 9 (Entity type modifiers function).

$$M_E : T_E \rightarrow \mathcal{P}(M)$$

, where $\mathcal{P}(M)$ is the power set of M .

An entity is something that is referenced in the requirement. Formally, entity is defined as a four-item tuple.

Definition 10 (Entity).

$$e = (n_e, t_e, m_e, d_e)$$

, where

- n_e is the name of the entity; $n_e \in N_E$
- t_e is the name of the entity; $t_e \in T_E$
- m_e is the set of modifiers of the entity; $m_e \in T_E$, $m_e \subseteq M_E(t_e)$
- d_e is the description of the entity; $d_e \in \Sigma^*$

3 Valuations and traces

Definition 11 (Valuation). A valuation is a function $\sigma : E \rightarrow V$ assigning a value to every entity. For entities of type *Event*, $\sigma(e) \in \{\text{true}, \text{false}\}$, where $\sigma(e) = \text{true}$ iff the event occurs at that time point.

We write Σ for the set of all valuations.

Definition 12 (Trace). A trace is a function $\rho : T \rightarrow \Sigma$. We write $\rho(t)(e)$ for the value of entity e at time t .

We write Σ^T for the set of all traces, i.e. the set of all functions from T to Σ . This is standard function space notation: $\Sigma^T = \{\rho \mid \rho : T \rightarrow \Sigma\}$.

4 Expressions and predicates

All expression functions defined in this section evaluate to a value in V . Functions in §4.1.2 depend only on time points or intervals. Functions in §4.1.3 additionally take a trace $\rho \in \Sigma^T$ and produce values that reflect the state of the system over time.

4.1 Expressions

4.1.1 Constants and arithmetic

Definition 13 (Constant). Any value $c \in V$ may appear as a constant expression.

The standard arithmetic operators $+$, $-$, $\times$, $/$ are defined on numeric values ($\mathbb{R}$ and its subsets) in the usual sense.

4.1.2 Time functions

The following functions operate on time points and intervals only; they do not depend on a trace.

The following two functions are defined for **discrete time only** ($T = T_D = \mathbb{N}$).

Definition 14. $Prev : T_D \rightarrow T_D$ returns the preceding time step:

$$Prev(t) = t - 1$$

The function is partial: it is undefined at $t = 0$ since $t - 1 \notin T_D$.

Definition 15. $Next : T_D \rightarrow T_D$ returns the following time step:

$$Next(t) = t + 1$$

Definition 16. $RelTime : T_D \times \mathbb{Z} \rightarrow T_D$ shifts a time point by n steps, where $n \in \mathbb{Z}$ may be positive or negative:

$$RelTime(t, n) = t + n$$

The function is partial: it is undefined when $t + n < 0$ since $t + n \notin T_D$. Note that $Prev(t) = RelTime(t, -1)$ and $Next(t) = RelTime(t, 1)$.

The following functions are defined for both time flavours.

Definition 17. $Diff : T \times T \rightarrow \Delta$ returns the duration between two time points:

$$Diff(t_1, t_2) = |t_1 - t_2|$$

Definition 18. $MkInterval : T \times T \rightarrow \mathcal{I}$ constructs an interval from a start and end time point:

$$MkInterval(t_1, t_2) = (t_1, t_2 - t_1)$$

The function is partial: it is undefined when $t_1 > t_2$.

Definition 19 (Since-zero interval). $SinceZero : T \rightarrow \mathcal{I}$ constructs an interval from time zero to a given time point t :

$$SinceZero(t) = (0, t)$$

Definition 20 (Interval start). $Start : \mathcal{I} \rightarrow T$ returns the start time point of an interval:

$$Start((t, d)) = t$$

Definition 21 (Interval end). $End : \mathcal{I} \rightarrow T$ returns the end time point of an interval:

$$End((t, d)) = t + d$$

Definition 22 (Interval duration). $Duration : \mathcal{I} \rightarrow \Delta$ returns the duration of an interval:

$$Duration((t, d)) = d$$

4.1.3 Trace functions

Expressions are terms that evaluate to a value in V given a trace ρ .

Definition 23. For a trace $\rho \in \Sigma^T$, the function $Val_\rho : E \times T \rightarrow V$ denotes the value of entity $e \in E$ at time $t \in T$:

$$Val_\rho(e, t) = \rho(t)(e)$$

Definition 24 (Event occurrence count). For a trace $\rho \in \Sigma^T$ and an entity $ev \in E$ of type **Event**, $EvtOccCount_\rho : E \times \mathcal{I} \rightarrow \mathbb{N}$ returns the number of occurrences of ev within an interval $i \in \mathcal{I}$:

$$EvtOccCount_\rho(ev, i) = |\{s \in T \mid Start(i) \leq s \leq End(i) \wedge Val_\rho(ev, s) = true\}|$$

Definition 25 (Last occurrences function). For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type **Event**, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \leq t_2 \leq \dots \leq t_n$ be the n most recent occurrence times of ev up to t , i.e. the n largest elements of $\{s \in T \mid s \leq t \wedge \text{Val}_\rho(ev, s) = \text{true}\}$ sorted in ascending order. Then $\text{LastOcc}_\rho : E \times T \times \mathbb{N}^+ \rightarrow \mathcal{I}$ is defined as:

$$\text{LastOcc}_\rho(ev, t, n) = \text{MkInterval}(t_1, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist up to t .

Note: The time of a single occurrence is obtained via $\text{LastOcc}_\rho(ev, t, 1)$, which returns an interval of duration 0. The occurrence time is then $\text{Start}(\text{LastOcc}_\rho(ev, t, 1))$.

Definition 26 (First occurrences function). For a trace $\rho \in \Sigma^T$, an entity $ev \in E$ of type **Event**, a time point $t \in T$, and $n \in \mathbb{N}^+$, let $t_1 \leq t_2 \leq \dots \leq t_n$ be the n earliest occurrence times of ev up to t , i.e. the n smallest elements of $\{s \in T \mid s \leq t \wedge \text{Val}_\rho(ev, s) = \text{true}\}$ sorted in ascending order. Then $\text{FirstOcc}_\rho : E \times T \times \mathbb{N}^+ \rightarrow \mathcal{I}$ is defined as:

$$\text{FirstOcc}_\rho(ev, t, n) = \text{MkInterval}(t_1, t_n)$$

The function is partial: it is undefined when fewer than n occurrences of ev exist up to t .

Note: The time of the first occurrence is $\text{Start}(\text{FirstOcc}_\rho(ev, t, 1))$.

Definition 27 (Maximum value function). For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $\text{MaxVal}_\rho : E \times \mathcal{I} \rightarrow V$ returns the maximum value of e over an interval $i \in \mathcal{I}$:

$$\text{MaxVal}_\rho(e, i) = \sup\{ \text{Val}_\rho(e, s) \mid s \in T, \text{Start}(i) \leq s \leq \text{End}(i) \}$$

Definition 28 (Minimum value function). For a trace $\rho \in \Sigma^T$ and an entity $e \in E$ with an ordered value domain, $\text{MinVal}_\rho : E \times \mathcal{I} \rightarrow V$ returns the minimum value of e over an interval $i \in \mathcal{I}$:

$$\text{MinVal}_\rho(e, i) = \inf\{ \text{Val}_\rho(e, s) \mid s \in T, \text{Start}(i) \leq s \leq \text{End}(i) \}$$

4.2 Predicates

A predicate is a function that evaluates to a value in $\mathbb{B}$. We write **Pred** for the set of all predicates.

Definition 29 (Boolean constants). $\top \in \mathbf{Pred}$ and $\perp \in \mathbf{Pred}$ are the always-true and always-false predicates respectively.

Definition 30 (Comparison predicate). For expressions $a, b \in V$ and a comparison operator $op \in \{=, \neq, <, \leq, >, \geq\}$:

$$\text{Cmp}(a, op, b) = a \text{ op } b$$

The operators $<, \leq, >, \geq$ require a, b to be from an ordered domain.

Definition 31 (Negation). For a predicate $P \in \mathbf{Pred}$:

$$\text{Not}(P) = \neg P$$

Definition 32 (Conjunction and disjunction). For a finite set of predicates $C \subseteq \mathbf{Pred}$:

$$\text{AllOf}(C) = \bigwedge_{P \in C} P \quad \text{AnyOf}(C) = \bigvee_{P \in C} P$$

Definition 33 (Implication). For predicates $A, B \in \mathbf{Pred}$:

$$\text{Implies}(A, B) = A \Rightarrow B$$

where $\Rightarrow$ is the standard logical implication.

Definition 34 (Quantifiers). For a finite set S and a predicate $P(x)$ parameterised by $x \in S$:

$$\text{ForAll}(S, P) = \bigwedge_{x \in S} P(x) \quad \text{Exists}(S, P) = \bigvee_{x \in S} P(x)$$

The following predicates operate on intervals and durations. Since $\Delta \subseteq \mathbb{R}$, duration comparison is a special case of *Cmp* (Definition 30).

Definition 35 (Interval inclusion). *Interval i_1 includes i_2 iff i_2 is entirely contained within i_1 :*

$$\text{IntervalIncludes}(i_1, i_2) \Leftrightarrow \text{Start}(i_1) \leq \text{Start}(i_2) \wedge \text{End}(i_2) \leq \text{End}(i_1)$$

Definition 36 (Interval exclusion). *Intervals i_1 and i_2 are exclusive iff they are completely disjoint:*

$$\text{IntervalExcludes}(i_1, i_2) \Leftrightarrow \text{End}(i_1) < \text{Start}(i_2) \vee \text{End}(i_2) < \text{Start}(i_1)$$

Definition 37 (Start included). *The start of i_1 falls within i_2 :*

$$\text{StartOfFirstIntervalIn}(i_1, i_2) \Leftrightarrow \text{Start}(i_2) \leq \text{Start}(i_1) \leq \text{End}(i_2)$$

Definition 38 (End included). *The end of i_1 falls within i_2 :*

$$\text{EndOfFirstIntervalIn}(i_1, i_2) \Leftrightarrow \text{Start}(i_2) \leq \text{End}(i_1) \leq \text{End}(i_2)$$

5 Operations

Definition 39 (Operation). *An operation is a function*

$$op : T \times A_1 \times \dots \times A_n \rightarrow \mathbf{Pred}$$

for some $n \geq 0$ and argument types $A_i \subseteq V \cup E$, where T is the time set. The predicate $op(t, a_1, \dots, a_n)$ is the effect of op executed at time t with arguments $a_1, \dots, a_n$, and must hold at time t . We write O for the set of all operations, and $O_T : T_E \rightarrow \mathcal{P}(O)$ for the function mapping each entity type to its applicable operations, where $O = \bigcup_{\tau \in T_E} O_T(\tau)$. The concrete mapping, including argument types and effects, is as follows.

Operation	Arguments	Effect $op(args)$
$calculate(t, e, expr)$	$t \in T$, <i>Signal</i> $e \in E$, value expression $expr \in V$	$Cmp(Val_\rho(e, t), =, expr)$
$read(t, e, a)$	$t \in T$, <i>Storage</i> $e \in E$, <i>address</i> a	$Cmp(Val_\rho(e, t), =, ValueAt(a))$
$store(t, e, dst)$	$t \in T$, <i>Storage</i> $e \in E$, <i>address</i> dst	$Cmp(ValueAt(dst), =, Val_\rho(e, t))$
$fire(t, e)$	$t \in T$, <i>Event</i> $e \in E$	$Cmp(Val_\rho(e, t), =, true)$
$transmit(t, e, v)$	$t \in T$, <i>Channel</i> $e \in E$, <i>value</i> $v \in V$	$Cmp(Val_\rho(e, t), =, v)$
$receive(t, e, v)$	$t \in T$, <i>Channel</i> $e \in E$, <i>value</i> $v \in V$	$Cmp(Val_\rho(e, t), =, v)$
$set_state(t, e)$	$t \in T$, <i>State</i> $e \in E$	$Cmp(Val_\rho(e, t), =, true)$
$compare(t, e_1, e_2)$	$t \in T$, <i>Entities</i> $e_1, e_2 \in E$	$Cmp(Val_\rho(e_1, t), =, Val_\rho(e_2, t))$

6 Notes

- Lepsi ponechat typ entity, protoze musime mapovat na modifiers, item ktere jsou rozdilne pro kazdy typ entity. Zaroven muzeme namapovat typ entity na mnozinu operaci.
- Modifiers need to be parameterized.
- We could theoretically map entity types directly to predicates, which can be set to *true*, but maybe we will want different predicates depending on the context (e.g. discrete or continuous time).
- T should be merged discrete and continuous.
- Need to resolve undefined with time functions.

- Last n-th occurrence.
- From time to predicates that hold? What would that be useful for? Do we really want to reference predicates in other predicates?